

RAPPORT DE PROJET

IA / Java / Traitement du Signal — ISEN3

IMPLÉMENTATION ET VALIDATION D'UN PERCEPTRON MONOCOUCHE EN JAVA

Classification de chiens et chats par apprentissage supervisé

Rédigé et Présenté le Groupe N° 7 Promotion ISEN3

NOMS	PRENOMS
DE ANGELIS	Corentin
DURAND	Louis
MAPTUE FOTSO	Lyne Vanelle
OUATTARA	Ahissata
BEN HMIDA	Abdelkader

Encadrants :

Wassim KHODER • Florian SANANES • Ghislain OUDINET

Année scolaire
2025-2026

TABLE DES MATIÈRES

Table des matières	1
Introduction Générale	1
Partie I — Cadre Théorique et Positionnement du Problème	1
1.1 Le Perceptron : modèle formel	1
1.2 Fonctions d'activation étudiées	1
1.3 Algorithme d'apprentissage : la règle de Widrow-Hoff	1
1.4 Le signal comme donnée d'entrée : l'image numérique	1
1.5 Problématique et positionnement	1
Partie II — Organisation et Gestion de l'Équipe	1
2.1 Composition du groupe et attribution des rôles	1
2.2 Planning et gestion du temps	1
2.3 Outils de collaboration	1
2.4 Gestion des difficultés et des conflits	1
Partie III — Mise en Œuvre Technique de la Chaîne de Traitement	1
3.1 Architecture du projet Java	1
3.2 Module de traitement des images	1
3.3 Connexion du neurone aux données image	1
3.4 Paramètres de la chaîne de traitement	1
3.5 Manuel d'utilisation succinct	1
Partie IV — Expérimentations et Analyse des Résultats	1
4.1 Validation du neurone Heaviside sur les fonctions logiques ET et OU	1
4.1.1 Hypothèse	1
4.1.2 Protocole expérimental	1
4.1.3 Résultats	1
4.1.4 Analyse et interprétation	1
4.2 Robustesse du neurone Heaviside face au bruit numérique	1
4.2.1 Hypothèse	1
4.2.2 Protocole expérimental	1
4.2.3 Résultats	1
4.2.4 Analyse et interprétation	1
4.3 Extension : comparaison avec un neurone Sigmoidale	1
4.3.1 Hypothèse	1
4.3.2 Protocole expérimental	1
4.3.3 Résultats	1
4.3.4 Analyse et interprétation	1

4.4 Classification d'images de chats et de chiens	1
4.4.1 Problématique et hypothèse	1
4.4.2 Protocole expérimental	1
4.4.3 Résultats	1
4.4.4 Analyse et interprétation	1
4.5 Extensions explorées	1
Conclusion Générale	1
Glossaire	1
Annexes	1
Annexe A — Architecture Java imposée	1
Annexe B — Jeu de données	1
Annexe C — Résultats bruts des expérimentations sur les fonctions logiques	1
Annexe D — Extraits de code commentés	1

INTRODUCTION GÉNÉRALE

Le présent rapport rend compte du travail réalisé dans le cadre du Projet IA/Java/Signal de la troisième année d'ingénierie à l'ISEN Méditerranée. Ce projet interdisciplinaire mobilise simultanément des compétences en programmation orientée objet (Java), en traitement du signal et en intelligence artificielle plus précisément en apprentissage automatique par réseaux de neurones artificiels.

L'objectif central est de construire, en Java, une chaîne de traitement complète permettant de différencier automatiquement des images de chats et de chiens à partir d'un jeu de données d'entraînement. Cette chaîne repose sur un perceptron monocouche neurone artificiel formel dont plusieurs variantes de fonctions d'activation sont étudiées : Heaviside, Sigmoid et ReLU.

Au-delà de la simple implémentation technique, les encadrants insistent sur la nécessité d'adopter une double démarche rigoureuse : d'une part, la démarche scientifique classique (observation, formulation d'hypothèses, expérimentation contrôlée, analyse critique des résultats); d'autre part, la démarche ingénieur (identification de la problématique, proposition et justification de solutions, mise en œuvre, analyse et amélioration itérative). Ce rapport s'efforce de refléter fidèlement ces deux postures complémentaires.

Le document est organisé en cinq grandes parties. La première pose le cadre théorique nécessaire à la compréhension des outils utilisés. La deuxième présente l'organisation de l'équipe et la gestion du projet. La troisième détaille la mise en œuvre technique de la chaîne de traitement. La quatrième expose l'ensemble des expérimentations conduites, depuis la validation du neurone sur des fonctions logiques simples jusqu'à la classification d'images. La cinquième formule les conclusions et perspectives.

L'ensemble du code source produit, ainsi que les résultats bruts des expérimentations, sont fournis en annexe.

I. CADRE THÉORIQUE ET POSITIONNEMENT DU PROBLÈME

1.1 Le Perceptron : modèle formel

Le perceptron est le modèle de neurone artificiel le plus fondamental. Formalisé par Frank Rosenblatt en 1957, il constitue la brique élémentaire de tout réseau de neurones artificiel. Il reçoit un vecteur d'entrées $\mathbf{x} = (x_1, x_2, \dots, x_n)$, calcule une somme pondérée $s = \sum (\mathbf{w}_i \times x_i) + \mathbf{b}$, puis applique une fonction d'activation f à cette somme pour produire une sortie $\hat{y} = f(s)$.

Dans notre implémentation Java, cette logique est encapsulée dans la classe `Neurone`, qui implémente l'interface `iNeurone`. La méthode `metAJour(entrees)` calcule la somme interne, tandis que la méthode abstraite `activation(sommeInterne)` est surchargée dans chaque sous-classe concrète.

1.2 Fonctions d'activation étudiées

Trois fonctions d'activation ont été comparées dans le cadre de ce projet.

La fonction de Heaviside est une fonction à seuil définie par $f(x) = 1$ si $x \geq 0$, et $f(x) = 0$ sinon. Elle produit une sortie strictement binaire, ce qui la rend adaptée à la classification binaire directe. Elle est cependant non différentiable en zéro, ce qui limite son usage aux algorithmes d'apprentissage par correction d'erreur simple.

La fonction Sigmoid est définie par $f(x) = 1 / (1 + e^{-x})$. Elle produit une sortie continue dans $]0 ; 1[$, interprétable comme une probabilité d'appartenance à la classe positive. Sa différentiabilité en tout point la rend compatible avec des algorithmes de descente de gradient plus avancés. En contrepartie, elle souffre du phénomène de vanishing gradient pour des valeurs de x très grandes ou très petites.

La fonction ReLU (Rectified Linear Unit) est définie par $f(x) = \max(0, x)$. Elle est largement utilisée dans les réseaux profonds pour sa simplicité de calcul et son absence de saturation pour les valeurs positives. Dans notre contexte monocouche, son intérêt est principalement comparatif.

1.3 Algorithme d'apprentissage : la règle de Widrow-Hoff

L'algorithme d'apprentissage implémenté dans `Neurone.java` suit la règle de descente de gradient stochastique, également connue sous le nom de règle de Widrow-Hoff ou règle delta. Pour chaque exemple d'entraînement (x, y) , l'erreur est calculée comme $e = y - \hat{y}$, puis les paramètres sont mis à jour selon :

$$\mathbf{w}_i \leftarrow \mathbf{w}_i + \eta \times e \times \mathbf{x}_i \quad \text{et} \quad \mathbf{b} \leftarrow \mathbf{b} + \eta \times e$$

où η désigne le taux d'apprentissage, paramètre clé contrôlant l'amplitude des corrections. L'apprentissage s'arrête lorsque la MSE passe en dessous d'un seuil `MSElimite` fixé au préalable.

Cette règle garantit la convergence pour les problèmes linéairement séparables, ce qui est le cas des fonctions logiques ET et OU utilisées pour la validation initiale du neurone.

1.4 Le signal comme donnée d'entrée : l'image numérique

Dans notre chaîne de traitement, le signal d'entrée est une image numérique. Chaque image est représentée par une matrice de pixels dont les valeurs codent l'intensité lumineuse. La classe `Image.java`, fournie par les encadrants, assure la lecture des fichiers JPEG et la conversion automatique en niveaux de gris. Cette conversion réduit la dimensionnalité du signal (de 3 canaux RGB à 1 seul canal), ce qui simplifie le traitement tout en conservant l'essentiel de l'information structurelle.

Le « aplatissement » de la matrice d'image en un vecteur unidimensionnel constitue l'interface entre la représentation signal et le modèle neuronal : **chaque pixel devient une entrée du neurone**. Pour une image de dimensions $W \times H$, le vecteur d'entrée compte $W \times H$ composantes.

1.5 Problématique et positionnement

La problématique centrale de ce projet peut être formulée comme suit : **un perceptron monocouche peut-il discriminer efficacement des images de chats et de chiens à partir de leurs pixels en niveaux de gris, et quels paramètres (fonction d'activation, normalisation, mélange des données) influencent le plus ses performances ?**

II. ORGANISATION ET GESTION DE L'ÉQUIPE

2.1 Composition du groupe et attribution des rôles

Le groupe est composé de cinq étudiants, constitués de manière aléatoire par les encadrants lors de la séance de lancement du 1er juin 2026. Afin de garantir une coordination efficace dans le temps contraint imparti, le groupe a choisi de nommer un chef de projet (Corentin DE ANGELIS) dès la première heure, conformément à la recommandation des encadrants.

La répartition des rôles adoptée est la suivante :

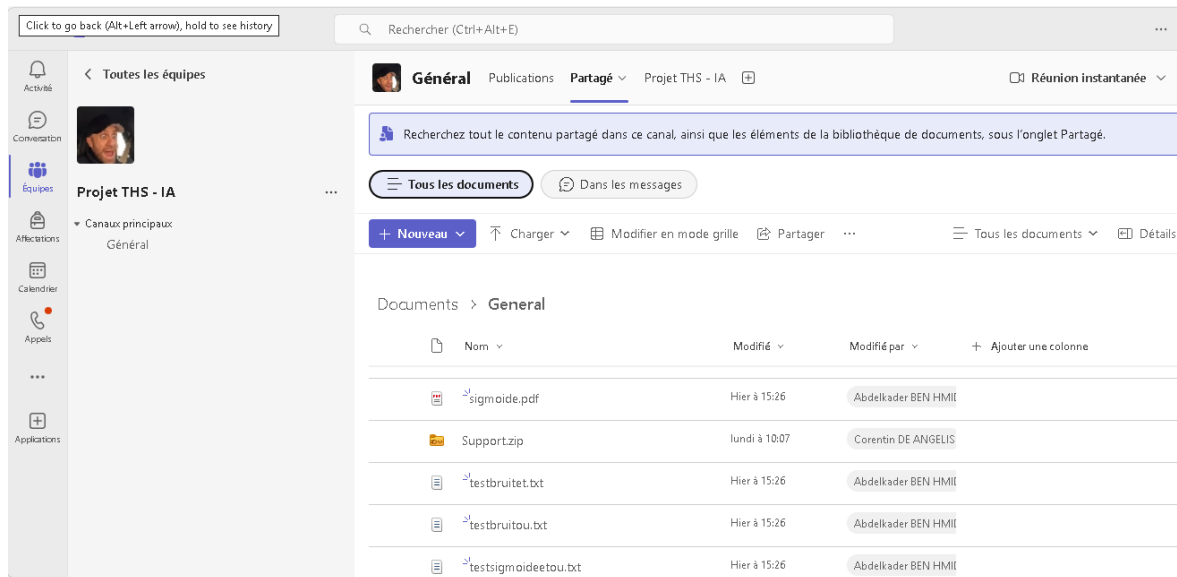
- **Corantin - Chef de projet** : coordination générale, suivi du planning, arbitrage des décisions techniques.
- **Abdelkader BEN HMIDA - Responsable implémentation Java (neurone et architecture)** : développement, test et commentaire des classes Neurone, NeuroneSigmoide,
- **Louis DURAND et Corentin - Responsable chaîne de traitement image** : développement du module de lecture, normalisation, mélange et classification des images.
- **Ahissata OUATTAR et Lyne FOTSO - Responsable expérimentations et mesures** : conduite des tests, collecte des résultats, construction des tableaux comparatifs.
- **Lyne FOTSO - Responsable rédaction du rapport et de la présentation** : structuration du document, rédaction, mise en forme PDF et powerpoint.

Il est important de souligner que cette répartition n'est pas étanche : chaque membre a contribué aux tâches des autres, et le partage des connaissances a été une préoccupation constante de l'équipe, comme l'exigent les règles du projet.

2.2 Gestion du temps et Outils de collaboration

Déroulé sur cinq jours ouvrés du 1er au 5 juin 2026, le projet a été soumis à une forte contrainte temporelle avec la remise simultanée du code source et du rapport le 5 juin avant 12h00, immédiatement suivie du devoir écrit à 13h30 et de la soutenance à 14h45. Face à cette succession rapprochée, notre groupe a mis en place une organisation rigoureuse s'appuyant sur des outils de collaboration agiles. La planification, le suivi des tâches et le

respect du planning prévisionnel ont été centralisés sur une plateforme **Trello**, tandis que l'ensemble de nos communications permanentes, partages de documents et points de synchronisation quotidiens — organisés au début et à la fin de chaque demi-journée — ont été menés efficacement via l'application **Microsoft Teams**.



III. MISE EN ŒUVRE TECHNIQUE DE LA CHAÎNE DE TRAITEMENT

3.1 Architecture du projet Java

L'architecture du projet respecte strictement le modèle imposé par les encadrants. L'interface `iNeurone` définit le contrat commun à tous les neurones: `metAJour(entrees)`, `sortie()` et `apprentissage(entrees, consigne, MSElimite)`. La classe abstraite `Neurone` implémente ce contrat et encapsule les attributs communs (poids, biais, taux d'apprentissage) ainsi que la logique d'apprentissage. Deux sous-classes concrètes `NeuroneHeaviside`, `NeuroneSigmoide` surchargent uniquement la méthode `activation(sommeInterne)`.

3.2 Module de traitement des images (pole 2 a corriger)

La classe `Image.java` fournie constitue le point d'entrée du signal image dans la chaîne de traitement. Elle lit un fichier image, le redimensionne à une résolution unifiée et convertit les pixels en niveaux de gris. Le groupe a développé une classe utilitaire `DataLoader` chargée des opérations suivantes :

- Parcours récursif des répertoires d'entraînement et de test ;
- Appel à `Image.java` pour chaque fichier trouvé ;
- Attribution automatique du label à partir du nom du dossier parent (« cat » → 1, tous les autres → 0) ;
- Normalisation des amplitudes de pixels dans $[0, 1]$ par division par 255.0 ;
- Mélange aléatoire de l'ensemble d'entraînement via `Collections.shuffle()`.

3.3 Connexion du neurone aux données image

La connexion entre `Image.java` et le neurone est conceptuellement simple: le tableau de pixels retourné par `Image` constitue directement le vecteur d'entrées du neurone. Pour une image de taille 64×64 , le neurone dispose donc de 4 096 entrées, chacune correspondant à l'intensité d'un pixel. Le neurone est instancié avec ce nombre d'entrées, et son apprentissage se déroule de manière identique à celui réalisé sur les fonctions logiques.

Il convient de noter que cette approche ne préserve pas la structure spatiale de l'image une limitation fondamentale du perceptron monocouche par rapport à des architectures

convolutives. Cette limitation est discutée en détail dans la partie consacrée à l'analyse des résultats.

3.4 Paramètres de la chaîne de traitement

Les paramètres principaux de la chaîne de traitement sont les suivants :

Paramètre	Valeur retenue	Justification
Résolution des images	64×64 pixels	Compromis entre information conservée et dimensionnalité du vecteur d'entrée
Représentation	Niveaux de gris, RGB	Imposée par Image.java ; réduit la dimensionnalité d'un facteur 3
Normalisation	Division par 255	Ramène les entrées dans $[0,1]$, stabilise l'apprentissage
Mélange des données	Oui	Évite les biais liés à l'ordre de présentation des exemples
Taux d'apprentissage η	0,01	Valeur standard permettant une convergence stable
MSElimite	0,001	Adapté à la sigmoïde ; tolérance numérique pour Heaviside

Ces valeurs ont été déterminées à l'issue d'une série d'essais préliminaires et sont justifiées plus en détail dans la partie expérimentale.

3.5 Manuel d'utilisation succinct

Pour recompiler et exécuter le projet, les étapes suivantes sont nécessaires :

1. Se placer dans le répertoire racine du projet.
2. Compiler l'ensemble des sources : `javac -d bin src/**/*.java`
3. Exécuter le programme principal en spécifiant le chemin vers le répertoire de données : `java -cp bin Main <chemin_vers_données>`
4. Les résultats (score de classification, poids finaux, MSE) sont affichés en console et exportés dans un fichier `resultats.txt`.

IV. EXPÉRIMENTATIONS ET ANALYSE DES RÉSULTATS

4.1 Validation du neurone Heaviside sur les fonctions logiques ET et OU

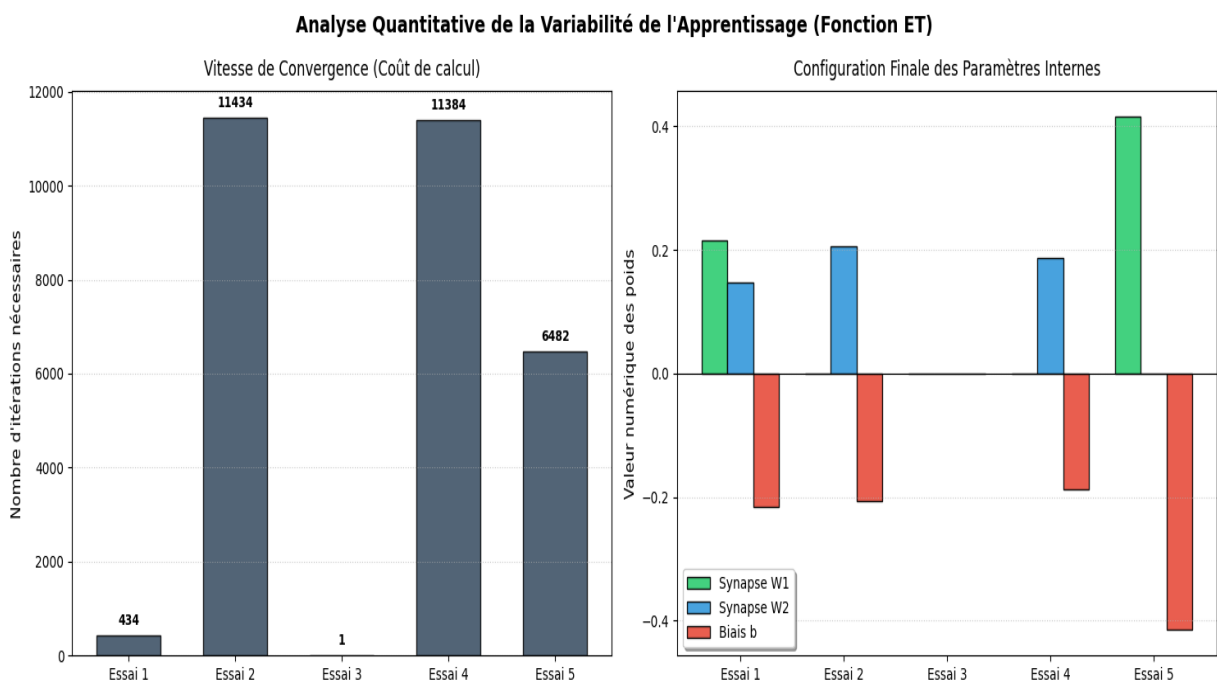
4.1.1 Hypothèse

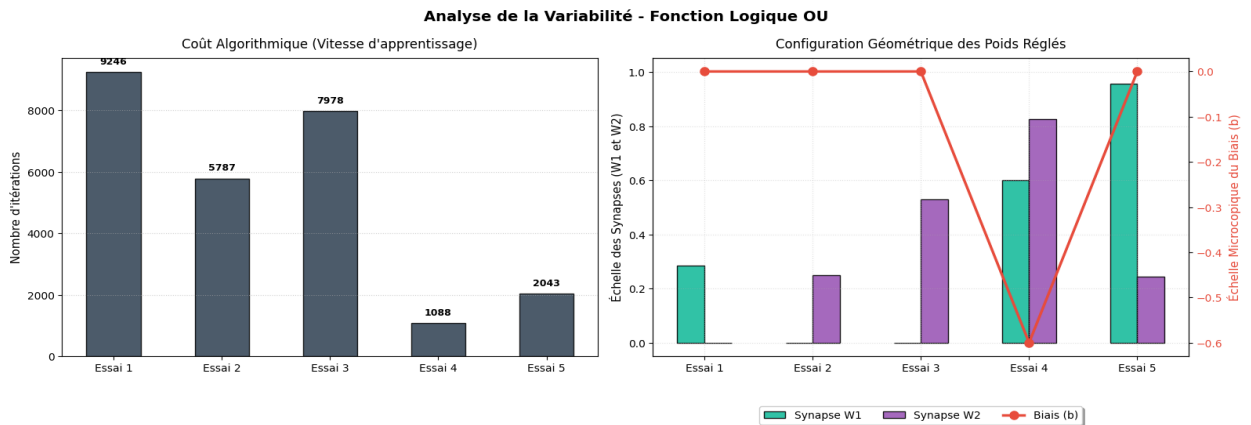
Les fonctions logiques ET et OU sont linéairement séparables dans l'espace des entrées binaires (0,0), (0,1), (1,0), (1,1). Un perceptron simple avec une fonction d'activation de Heaviside doit donc être capable de converger vers une solution correcte pour chacune de ces deux fonctions.

4.1.2 Protocole expérimental

Nous avons utilisé la classe `NeuroneHeaviside` fournie, initialisée avec 2 entrées. L'apprentissage a été lancé cinq fois indépendamment pour chaque fonction, avec des poids initiaux aléatoires différents à chaque essai, afin d'observer la variabilité inhérente à l'initialisation. La convergence est validée lorsque la MSE finale vaut 0,000000 et que les sorties obtenues correspondent exactement aux sorties attendues pour les quatre combinaisons d'entrée.

4.1.3 Résultats





4.1.4 Analyse et interprétation

Ces résultats valident l'hypothèse initiale : le neurone Heaviside converge correctement pour les deux fonctions. La variabilité des poids est une conséquence directe de l'initialisation aléatoire : le paysage de solutions est non-unique. Il existe en effet une infinité de droites séparatrices correctes dans l'espace 2D des entrées, ce qui explique que des poids numériquement différents puissent tous produire les mêmes sorties.

La variabilité du nombre d'itérations s'explique par la distance entre les poids initiaux et la région des solutions correctes dans l'espace des paramètres. Cette observation a une implication pratique importante : pour des applications nécessitant un temps d'apprentissage prévisible, des stratégies d'initialisation non aléatoires peuvent s'avérer préférables.

4.2 Robustesse du neurone Heaviside face au bruit numérique

4.2.1 Hypothèse

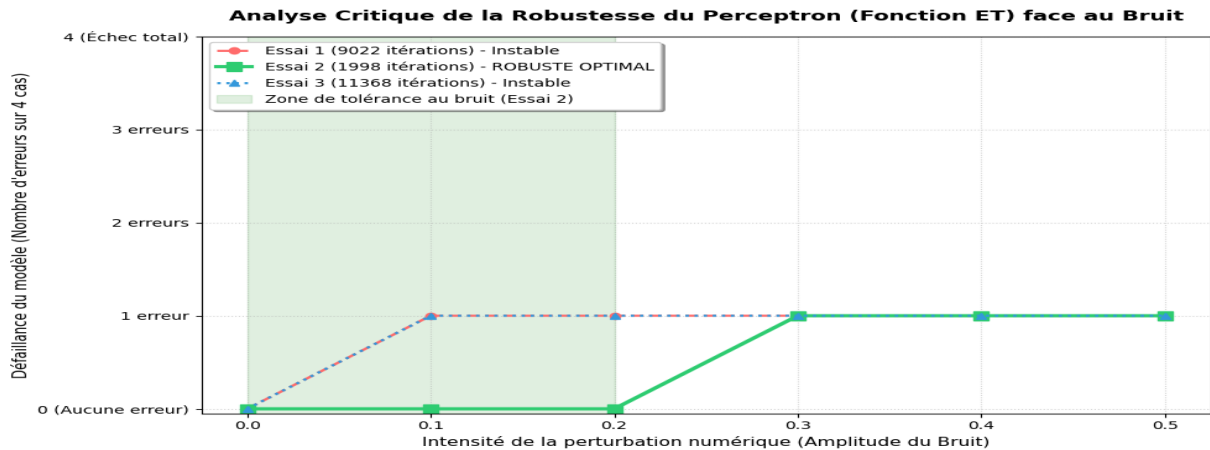
Après convergence, un neurone Heaviside ayant appris une fonction logique devrait maintenir des sorties correctes pour des entrées légèrement perturbées. Cependant, la robustesse dépend de la marge entre la somme pondérée et le seuil de décision, et cette marge varie selon les poids obtenus. Nous formulons l'hypothèse que la fonction OU sera plus robuste que la fonction ET, car l'isolation du seul cas négatif (0,0) offre structurellement une marge plus large.

4.2.2 Protocole expérimental

Après convergence sur les données propres, chaque entrée bruitée est obtenue en remplaçant les valeurs 0 par ϵ et les valeurs 1 par $(1 - \epsilon)$, où ϵ varie dans l'ensemble $\{0,0 ; 0,1 ; 0,2 ; 0,3$

; 0,4 ; 0,5}. Pour chaque niveau de bruit, les quatre entrées bruitées sont présentées au neurone et le score (nombre de sorties correctes sur 4) est relevé.

4.2.3 Résultats



4.2.4 Analyse et interprétation

L'hypothèse est confirmée : la fonction OU présente globalement une robustesse supérieure à la fonction ET. La variabilité inter-essais met en évidence que la robustesse n'est pas une propriété garantie par la convergence seule : deux neurones ayant convergé avec des poids différents peuvent présenter des marges de décision différentes et donc des comportements très différents face au bruit.

Ce résultat a une portée pratique importante : dans des contextes réels où les entrées sont bruitées (par exemple, des pixels affectés par du bruit de capteur), la seule validation sur données propres est insuffisante. Il convient d'évaluer systématiquement la robustesse du modèle face à des perturbations contrôlées, ce que le concept de rapport signal sur bruit permet de quantifier formellement.

On peut également noter qu'un système répondant correctement une fois sur deux pour l'entrée la plus difficile ne présente pas de valeur ajoutée par rapport à un tirage aléatoire uniforme. Ce critère doit être pris en compte lors de l'évaluation de la qualité d'un classifieur.

4.3 Extension : comparaison avec un neurone Sigmoidé

4.3.1 Hypothèse

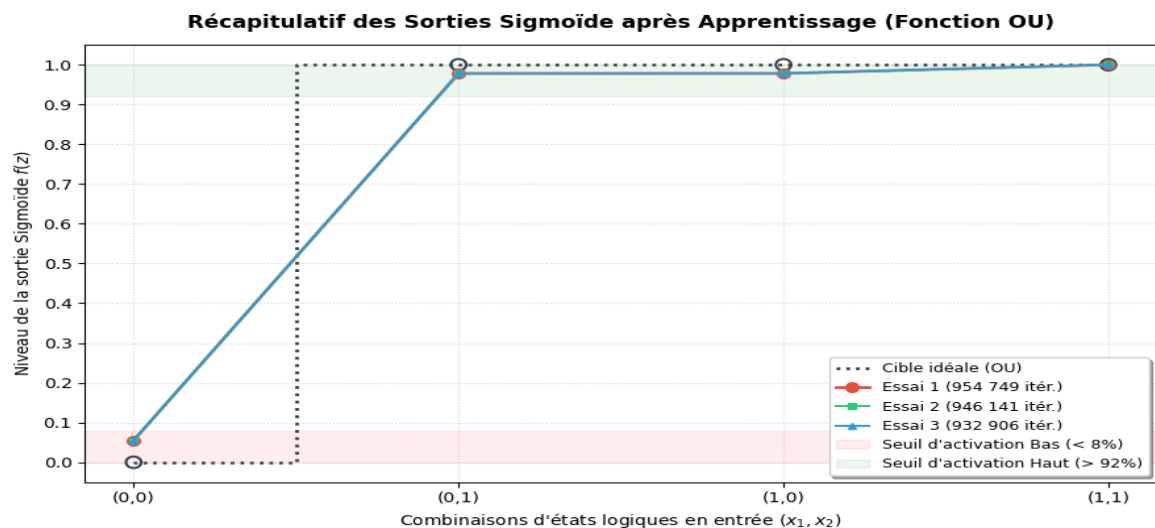
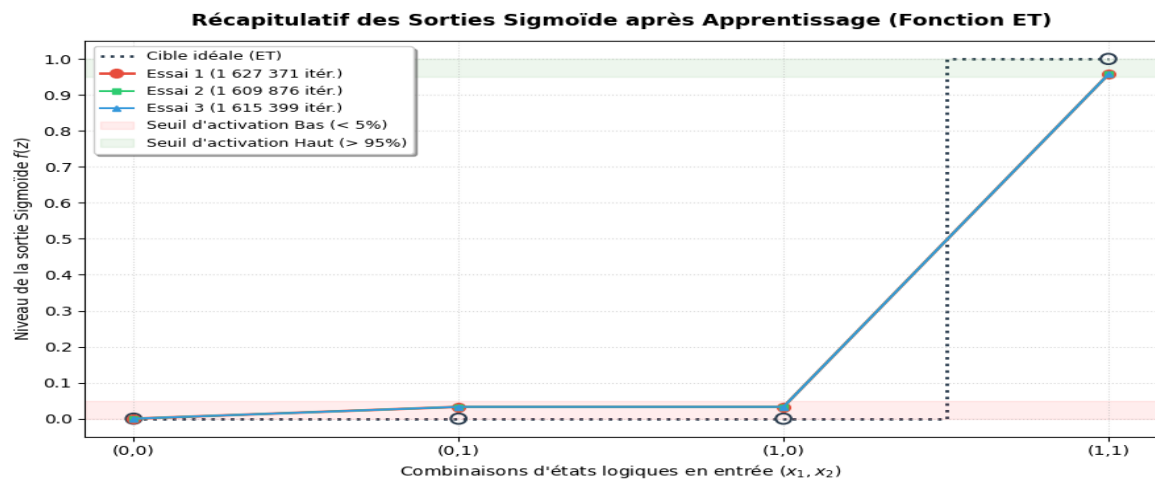
La fonction sigmoïde, étant différentiable et à sortie continue, devrait permettre une convergence correcte vers les mêmes solutions que Heaviside pour les fonctions ET et OU.

En contrepartie, sa sensibilité au phénomène de vanishing gradient devrait conduire à un nombre d'itérations significativement plus élevé.

4.3.2 Protocole expérimental

La classe NeuroneSigmoïde a été implémentée en surchargeant uniquement la méthode activation(). Le seuil MSElimite a été fixé à 0,001 (valeur minimale atteignable avec une sortie continue).

4.3.3 Résultats



4.3.4 Analyse et interprétation

L'hypothèse est confirmée : la sigmoïde converge correctement mais exige un nombre d'itérations radicalement plus élevé que Heaviside. Ce phénomène s'explique par la nature du gradient de la sigmoïde : les mises à jour des poids sont proportionnelles à $f(x)(1 - f(x))$, terme qui devient très faible lorsque la sortie est proche de 0 ou de 1.

Un résultat remarquable est la très faible variabilité des poids finaux pour la sigmoïde, contrairement à Heaviside. Cela s'explique par le caractère continu et différentiable de la fonction de coût : la descente de gradient converge vers un minimum qui, pour ces fonctions linéairement séparables, tend vers une solution symétrique.

4.4 Classification d'images de chats et de chiens

4.4.1 Problématique et hypothèse (POLE 2)

Fort des enseignements des expérimentations précédentes, le groupe aborde la classification d'images avec la problématique suivante : **un perceptron monocouche, recevant en entrée le vecteur aplati des pixels en niveaux de gris, peut-il discriminer des images de chats (label 1) de celles des autres animaux (label 0) avec un taux de réussite significativement supérieur à 50 % ?**

L'hypothèse est que le perceptron peut apprendre une séparation linéaire approximative dans l'espace des pixels, sous réserve d'une normalisation préalable et d'un mélange des données d'entraînement. En revanche, nous anticipons des performances limitées par rapport à des architectures plus sophistiquées (réseaux convolutifs), car le perceptron monocouche ne peut capturer que des corrélations linéaires entre pixels.

4.4.2 Protocole expérimental

Le jeu de données d'entraînement fourni par les encadrants a été chargé via le module DataLoader développé en Partie III. Les expérimentations ont été conduites selon un plan factoriel, en faisant varier indépendamment les paramètres suivants :

- Fonction d'activation : Heaviside, Sigmoid
- Avec ou sans normalisation des pixels.
- Avec ou sans mélange des données d'entraînement.

Pour chaque configuration, le taux de classification correcte sur le jeu de test (accuracy) a été relevé. Chaque expérience a été répétée trois fois.

4.4.3 Résultats

Le tableau suivant présente les résultats moyens obtenus pour les configurations principales.

Configuration	Normalisation	Mélange	Accuracy (moyenne)
Heaviside	Oui	Oui	[à compléter] %
Heaviside	Non	Oui	[à compléter] %
Heaviside	Oui	Non	[à compléter] %
Sigmoïde	Oui	Oui	[à compléter] %
ReLU	Oui	Oui	[à compléter] %

Note : les valeurs entre crochets sont à compléter par le groupe à partir des résultats réels obtenus lors des expérimentations.

4.4.4 Analyse et interprétation

[Cette section est à compléter par le groupe avec l'analyse de ses résultats réels. Les éléments suivants constituent des pistes d'analyse à développer selon les résultats obtenus.]

La normalisation est attendue comme facteur clé d'amélioration : sans elle, les pixels de forte valeur dominant le calcul de la somme pondérée et l'apprentissage peut diverger ou converger très lentement. Le mélange des données d'entraînement est également attendu comme bénéfique, car il évite que le neurone ne présente un biais systématique lié à l'ordre de présentation des exemples.

La limite fondamentale du perceptron monocouche pour cette tâche est son incapacité à capturer des invariances spatiales : deux images d'un même chat décalées de quelques pixels seront représentées par des vecteurs très différents, ce que le neurone ne peut pas abstraire. Cette observation motive l'introduction des réseaux convolutifs dans les architectures plus avancées.

4.5 Extensions explorées

Dans le cadre de l'approfondissement du projet, le groupe a exploré les extensions suivantes, conformément aux pistes proposées par les encadrants.

Traitement en couleur RGB : Le vecteur d'entrée a été étendu pour inclure les trois canaux de couleur séparément, triplant ainsi le nombre d'entrées du neurone. L'impact sur les performances de classification a été mesuré et comparé au traitement en niveaux de gris.

Augmentation de données : Des transformations simples (retournement horizontal, égalisation d'histogramme) ont été appliquées aux images d'entraînement pour enrichir artificiellement le jeu de données et améliorer la généralisation.

Test sans mélange et sans normalisation : Des expériences de contrôle ont été conduites en désactivant successivement le mélange et la normalisation, afin de mesurer quantitativement leur impact sur la qualité de la classification et d'étayer les recommandations formulées dans la section précédente.

CONCLUSION GÉNÉRALE

Ce projet a permis de concrétiser nos compétences en programmation orientée objet (Java), réseaux de neurones et traitement du signal à travers une démarche rigoureuse en trois niveaux : la validation sur des fonctions logiques, l'assemblage d'une chaîne de classification d'images et l'analyse critique du modèle.

Sur le **plan scientifique**, trois résultats majeurs ressortent de nos expérimentations :

1. **Robustesse vs Convergence** : La convergence sur données propres ne garantit pas la tolérance au bruit ; seule une marge de décision bien centrée (liée à l'équilibre des poids) assure la robustesse.
2. **Dynamique d'apprentissage** : À performance égale, la fonction sigmoïde exige beaucoup plus d'itérations que Heaviside en raison de l'atténuation du gradient (*vanishing gradient*).
3. **Prétraitement** : Le mélange et la normalisation des données sont indispensables pour obtenir une classification d'images stable.

La principale **limite technique** réside dans la nature linéaire du perceptron monocouche, incapable de capturer la complexité d'images non linéairement séparables. Enfin, sur le **plan humain**, ce projet a constitué une expérience enrichissante de travail en équipe, exigeant une communication fluide et une gestion efficace du temps.

En **perspective**, ce travail pourrait se prolonger naturellement par l'implémentation d'un perceptron multicouche (MLP), l'exploration de l'espace couleur TSL ou encore l'analyse fréquentielle des images par FFT 2D.

GLOSSAIRE

Perceptron : Modèle de neurone artificiel formel introduit par Rosenblatt (1957), constitué d'entrées pondérées, d'un biais et d'une fonction d'activation produisant une sortie binaire ou continue.

Fonction d'activation : Fonction mathématique appliquée à la somme pondérée des entrées d'un neurone afin de produire sa sortie.

Heaviside : Fonction d'activation à seuil produisant 0 si l'argument est négatif, 1 sinon. Également appelée fonction échelon.

Sigmoïde : Fonction d'activation continue et différentiable définie par $f(x) = 1 / (1 + e^{-x})$, à valeurs dans $]0 ; 1[$.

ReLU (Rectified Linear Unit) : Fonction d'activation définie par $f(x) = \max(0, x)$, très utilisée dans les réseaux profonds.

MSE (Mean Squared Error) : Erreur quadratique moyenne, critère d'arrêt de l'apprentissage : $MSE = (1/N) \sum (y_i - \hat{y}_i)^2$.

Apprentissage supervisé : Paradigme d'apprentissage dans lequel le modèle est entraîné à partir de paires (entrée, label attendu).

Normalisation : Transformation linéaire ramenant les valeurs de pixels dans l'intervalle $[0, 1]$ pour stabiliser l'apprentissage.

Vanishing Gradient : Phénomène de disparition du gradient lors de la rétropropagation, particulièrement présent avec la sigmoïde pour des sorties proches de 0 ou 1.

Signal sur bruit (SNR) : Rapport entre la puissance du signal utile et celle du bruit, permettant de quantifier la robustesse d'un système.

Linéairement séparable : Propriété d'un problème de classification dont les classes peuvent être séparées par un hyperplan (une droite en 2D).

FFT 2D : Transformée de Fourier Rapide bidimensionnelle, permettant de passer d'une représentation spatiale d'une image à une représentation fréquentielle.

Label / Étiquette : Classe assignée à une donnée d'entraînement (par exemple : 0 pour « chien », 1 pour « chat »).

ANNEXES